- LINGESH.S
- (1924260290)

TASK -1

```
#include <iostream>
#include <string>
#include <vector>


enum class Zone { Inner, Middle, Outer, Airport };

class SmartCard {
private:
long long balancePaise; // Stored in paise to avoid rounding errors (Task 3 requirement)
std::string cardID;
bool isStudent;
bool isSenior;

// Constant for the safety rule: Balance must stay above Rs 50
static constexpr long long MIN_REMAINING_BALANCE = 5000;

public:
SmartCard(std::string id, long long initialBalance, bool student, bool senior)
: cardID(id), balancePaise(initialBalance), isStudent(student), isSenior(senior) {}

// Behavior: Check if the card can afford the journey
bool canAfford(long long farePaise) const {
return (balancePaise - farePaise) >= MIN_REMAINING_BALANCE;
}

// Behavior: Deduct fare (Encapsulation: only this method can change balance)
void deductFare(long long farePaise) {
if (canAfford(farePaise)) {
balancePaise -= farePaise;
std::cout << "Journey successful. New balance: Rs " << balancePaise / 100.0 << "\n";
} else {
std::cout << "Transaction Denied: Insufficient funds to maintain minimum reserve.\n";
}
}
```

- LINGESH.S
- (1924260290)

```
// Getters for abstraction
long long getBalance() const { return balancePaise; }
bool getIsStudent() const { return isStudent; }
bool getIsSenior() const { return isSenior; }
};

// --- OBJECT: FareTable ---
// Demonstrates ABSTRACTION: The user doesn't need to know how math is done
class FareTable {
public:
// Logic for fare calculation based on zone and peak status
static long long calculateFare(Zone z, bool isPeak, bool isStudent, bool isSenior) {
long long baseFare = 0;

// Selection logic (Task 6 requirement)
switch (z) {
case Zone::Inner: baseFare = 1000; break;
case Zone::Middle: baseFare = 2000; break;
case Zone::Outer: baseFare = 3000; break;
default: baseFare = 0;
}

// Apply Peak Surcharge (20%)
if (isPeak) {
baseFare += (baseFare * 20) / 100;
}

// Apply Concessions (Task 1 requirement)
if (isStudent || isSenior) {
baseFare = (baseFare * 80) / 100; // Example: 20% discount
}

return baseFare;
}
};

// --- OBJECT: Journey ---
```

- LINGESH.S
- (1924260290)

```
class Journey {
public:
// Behavior: Links the Card and the FareTable
static void process(SmartCard& card, Zone zone, bool isPeak) {
long long fare = FareTable::calculateFare(zone, isPeak, card.getIsStudent(), card.getIsSenior());

std::cout << "Attempting journey... Calculated Fare: Rs " << fare / 100.0 << "\n";
card.deductFare(fare);
}
};

// --- MAIN: Testing the Design ---
int main() {
// Creating a SmartCard object for a student
SmartCard myCard("CARD123", 10000, true, false); // Rs 100.00

std::cout << "--- Starting Metro System Simulation ---\n";

// Journey 1: Inner Zone, Off-Peak
std::cout << "\nJourney 1 (Inner, Off-Peak):\n";
Journey::process(myCard, Zone::Inner, false);

// Journey 2: Outer Zone, Peak Hour
std::cout << "\nJourney 2 (Outer, Peak):\n";
Journey::process(myCard, Zone::Outer, true);

// Journey 3: Trying to travel when balance is too low
std::cout << "\nJourney 3 (Outer, Peak - Should be denied):\n";
Journey::process(myCard, Zone::Outer, true);

return 0;
}
```

- LINGESH.S
- (1924260290)

TASK -2

```
#include <iostream>
using namespace std;

// Constants: define values in one place only
constexpr double FARE_PER_ZONE = 20.0;
constexpr double SURCHARGE = 10.0;
constexpr double MIN_BALANCE = 50.0;
constexpr double INITIAL_BALANCE = 200.0;

// enum class safely represents travel zones
enum class Zone {
   ONE = 1,
   TWO = 2,
   THREE = 3,
   FOUR = 4
};

// SmartCard class
class SmartCard {
private:
   int cardNumber;
   double balance;
```

- LINGESH.S
- (1924260290)

```
public:
    SmartCard(int number, double initialBalance)
        : cardNumber(number), balance(initialBalance) {}

    void travel(Zone zone, bool applySurcharge = false) {
        int zones = static_cast<int>(zone);

        if (zones <= 0) {
            cout << "Invalid zone.\n";
            return;
        }

        double fare = zones * FARE_PER_ZONE;

        if (applySurcharge) {
            fare += SURCHARGE;
        }

        if (balance - fare >= MIN_BALANCE) {
            balance -= fare;

            cout << "Card Number: " << cardNumber << '\n';
            cout << "Fare: Rs. " << fare << '\n';
            cout << "Journey successful.\n";
            cout << "Remaining Balance: Rs. "
                 << balance << '\n';
```

```
        } else {
            cout << "Journey rejected: insufficient balance.\n";
            cout << "Balance: Rs. " << balance << '\n';
        }
    }
};

int main() {
    SmartCard card(101, INITIAL_BALANCE);

    cout << "Journey 1:\n";
    card.travel(Zone::THREE);

    cout << "\nJourney 2:\n";
    card.travel(Zone::TWO, true);

    return 0;
}
```

TASK -3

```
#include <iostream>
#include <iomanip>
#include <climits>
#include <limits>
```

- LINGESH.S
- (1924260290)

```
#include <string>
using namespace std;

int main() {
   // Suitable data types for a fare collection system
   bool paymentSuccessful = true;
   char zoneCode = 'A';
   int zoneNumber = 5;
   long long farePaise = 25000LL;       // Rs. 250.00
   long long balancePaise = 100000LL;   // Rs. 1000.00
   unsigned int journeyCount = 500;
   double journeyTime = 12.5;           // Minutes
   float averageSpeed = 35.5f;

   cout << "DATA TYPES AND SIZE\n";
   cout << "bool: " << sizeof(bool) << " bytes\n";
   cout << "char: " << sizeof(char) << " bytes\n";
   cout << "int: " << sizeof(int) << " bytes\n";
   cout << "long long: " << sizeof(long long) << " bytes\n";
   cout << "unsigned int: " << sizeof(unsigned int) << " bytes\n";
   cout << "float: " << sizeof(float) << " bytes\n";
   cout << "double: " << sizeof(double) << " bytes\n";
   cout << "auto (fare): " << sizeof(farePaise) << " bytes\n\n";

   cout << "FARE DETAILS\n";
   cout << "Fare: " << farePaise / 100 << "."
```

```
    << setw(2) << setfill('0') << farePaise % 100
    << " rupees\n";
cout << setfill(' ');
cout << "Balance in paise: " << balancePaise << '\n';
cout << "Zone code: " << zoneCode << '\n';
cout << "Zone number: " << zoneNumber << '\n';
cout << "Journey count: " << journeyCount << '\n';
cout << "Journey time: " << journeyTime << " minutes\n";
cout << "Average speed: " << averageSpeed << '\n';
cout << "Payment successful: " << boolalpha
    << paymentSuccessful << "\n\n";

// Money precision comparison
double rupees = 0.10;
double total = 0.0;

for (int i = 0; i < 10; i++) {
   total += rupees;
}

long long totalPaise = 0;
for (int i = 0; i < 10; i++) {
   totalPaise += 10;
}

cout << "MONEY PRECISION\n";
```

```
cout << setprecision(17);
cout << "Double total: " << total << " rupees\n";
cout << "Integer total: " << totalPaise
    << " paise = " << totalPaise / 100.0
    << " rupees\n\n";

// Type compatibility
int a = 10;
double b = 2.5;
bool c = true;

double result = a + b + c;

cout << "TYPE COMPATIBILITY\n";
cout << "int + double + bool = " << result << '\n';
cout << "bool true converts to 1 in arithmetic.\n\n";

// Limits from climits and limits
cout << "TYPE RANGES\n";
cout << "INT_MIN: " << INT_MIN << '\n';
cout << "INT_MAX: " << INT_MAX << '\n';
cout << "LLONG_MIN: " << LLONG_MIN << '\n';
cout << "LLONG_MAX: " << LLONG_MAX << '\n';
cout << "UINT_MAX: " << UINT_MAX << '\n';
cout << "Float digits: "
    << numeric_limits<float>::digits10 << '\n';
```

```
    cout << "Double digits: "
         << numeric_limits<double>::digits10 << '\n';

    // Safe overflow check before addition
    long long collectionPaise = 2000000LL;
    long long nextFarePaise = 50000LL;

    cout << "\nOVERFLOW CHECK\n";
    if (collectionPaise <= LLONG_MAX - nextFarePaise) {
        collectionPaise += nextFarePaise;
        cout << "Updated collection: "
             << collectionPaise << " paise\n";
    } else {
        cout << "Overflow detected!\n";
    }

    return 0;
}
```

TASK -4

```
#include <iostream>
using namespace std;
```

- LINGESH.S
- (1924260290)

```
int main()
{
    int a = 7, b = 2;

    // Integer division and double conversion
    double c = a / b;
    double d = static_cast<double>(a) / b;

    cout << c << ' ' << d << '\n';

    // Character to integer promotion
    char ch = 'A' + 2;
    cout << ch << ' ' << ch + 1 << '\n';

    // Integer to bool conversion
    bool flag = 5;
    cout << flag << ' ' << boolalpha << flag << '\n';

    // Signed and unsigned comparison
    unsigned int u = 1;
    int s = -1;
    cout << (s < u ? "s < u" : "s >= u") << '\n';

    return 0;
}
```

- LINGESH.S
- (1924260290)

TASK -5

```
#include <iostream>
using namespace std;

int main()
{
   int x = 5, y = 3;

   // Operator precedence
   cout << x + y * 2 << ' ' << (x + y) * 2 << '\n';

   // Increment operators and associativity
   int z = x++ + ++y;
   cout << z << ' ' << x << ' ' << y << '\n';

   // Conditional operator and modulus
   cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\n';

   // Integer division and negative operands
   cout << 10 / 3 << ' ' << 10 % 3 << ' '
       << -10 / 3 << ' ' << -10 % 3 << '\n';

   // Compound assignment
   int k = 0;
   for (int i = 0; i < 10; i += 3)
```

```
    k += i;

  cout << k << '\n';

  // Peak-hour check: morning and evening
  int hour = 18;
  bool peak = (hour >= 8 && hour < 10) ||
          (hour >= 17 && hour < 20);

  cout << "Peak hour: " << (peak ? "Yes" : "No") << '\n';

  // Surcharge and concession examples
  double fare = 100.0;
  bool isPeak = true;
  bool concession = false;

  double finalFare = fare * (isPeak ? 1.2 : 1.0);
  finalFare *= (concession ? 0.5 : 1.0);

  cout << "Final fare: " << finalFare << '\n';

  return 0;
}
```

TASK -6

- LINGESH.S
- (1924260290)

```
#include <iostream>
#include <iomanip>
using namespace std;

enum class Zone { Inner, Middle, Outer };

constexpr int BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;

// Calculate fare using a switch statement
long long fare(Zone z, bool peak)
{
   long long base = 0;

   switch (z)
   {
      case Zone::Inner:
         base = BASE_PAISE[0];
         break;

      case Zone::Middle:
         base = BASE_PAISE[1];
         break;
```

```
        case Zone::Outer:
            base = BASE_PAISE[2];
            break;

        default:
            return 0;
    }

    if (peak)
    {
        base += (base * PEAK_SURCHARGE_PERCENT) / 100;
    }

    return base;
}

// Check minimum remaining balance
bool canTravel(long long balance, long long f)
{
    return balance >= f &&
           balance - f >= MIN_BALANCE_PAISE;
}

int main()
{
    long long balance = 10000; // Rs. 100.00
```

- LINGESH.S
- (1924260290)

```
struct Trip
{
   Zone zone;
   bool peak;
};

Trip trips[] = {
   {Zone::Inner, false},
   {Zone::Outer, true},
   {Zone::Middle, false}
};

// Process all journeys
for (const Trip& t : trips)
{
   long long f = fare(t.zone, t.peak);

   if (canTravel(balance, f))
   {
      balance -= f;

      cout << "Fare Rs "
         << f / 100 << "."
         << setw(2) << setfill('0') << f % 100
         << setfill(' ');
```

```
    }
    else
    {
        cout << "Denied (low balance)";
    }

    cout << " balance Rs " << balance / 100 << '\n';
  }

  // Check peak-hour boundaries
  auto isPeakHour = [](int hour)
  {
    return (hour >= 8 && hour < 10) ||
           (hour >= 17 && hour < 20);
  };

  cout << boolalpha;
  cout << "10 AM peak: " << isPeakHour(10) << '\n';
  cout << "8 PM peak: " << isPeakHour(20) << '\n';

  return 0;
}
```

TASK -7

- LINGESH.S
- (1924260290)

```
#include <iostream>
using namespace std;

const long long MIN_BALANCE = 5000; // Rs. 50 in paise

long long calculateFare(int zone, bool peak)
{
   long long baseFare;

   switch (zone)
   {
      case 1: baseFare = 1000; break; // Rs. 10
      case 2: baseFare = 2000; break; // Rs. 20
      case 3: baseFare = 3000; break; // Rs. 30
      default: return -1;
   }

   if (peak)
      baseFare += baseFare * 20 / 100;

   return baseFare;
}

bool canTravel(long long balance, long long fare)
{
   return fare >= 0 &&
```

```
        balance >= fare &&
        balance - fare >= MIN_BALANCE;
}

bool deduct(long long &balance, long long fare)
{
   if (!canTravel(balance, fare))
      return false;

   balance -= fare;
   return true;
}

int main()
{
   long long balance = 10000; // Rs. 100
   int zone[] = {1, 3, 2};
   bool peak[] = {false, true, false};

   for (int i = 0; i < 3; i++)
   {
      long long fare = calculateFare(zone[i], peak[i]);

      if (deduct(balance, fare))
         cout << "Travel approved. ";
      else
```

```
        cout << "Travel denied. ";

    cout << "Balance Rs " << balance / 100 << '\n';
  }

  return 0;
}
```

TASK -8

```
#include <iostream>
#include <iomanip>
#include <limits>
#include <string>
using namespace std;

int main()
{
  string passengerName;
  int journeys;
  long long farePaise;
  long long totalPaise = 0;

  cout << "===== SMART GATE SYSTEM =====\n";

  cout << "Enter passenger name: ";
```

```
getline(cin, passengerName);

// Safely read the number of journeys
while (true)
{
    cout << "Enter number of journeys: ";
    cin >> journeys;

    if (cin.fail())
    {
        cout << "Invalid input! Enter a number.\n";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        continue;
    }

    if (journeys <= 0)
    {
        cout << "Journeys must be greater than zero.\n";
        continue;
    }

    break;
}

// Read fare for each journey
```

- LINGESH.S
- (1924260290)

```
for (int i = 1; i <= journeys; i++)
{
   while (true)
   {
      cout << "Enter fare for journey " << i
          << " in paise: ";
      cin >> farePaise;

      if (cin.fail())
      {
         cout << "Invalid input! Enter an integer.\n";
         cin.clear();
         cin.ignore(
            numeric_limits<streamsize>::max(), '\n');
         continue;
      }

      if (farePaise < 0 ||
         farePaise > numeric_limits<long long>::max()
                     - totalPaise)
      {
         cout << "Invalid fare or total overflow.\n";
         continue;
      }

      totalPaise += farePaise;
```

```
            break;
        }
    }

    // Print a formatted report
    cout << "\n========== JOURNEY REPORT ==========\n";
    cout << left << setw(22) << "Passenger"
         << ": " << passengerName << '\n';

    cout << left << setw(22) << "Total journeys"
         << ": " << journeys << '\n';

    cout << left << setw(22) << "Total fare"
         << ": Rs " << fixed << setprecision(2)
         << static_cast<double>(totalPaise) / 100.0
         << '\n';

    cout << "====================================\n";

    return 0;
}
```

TASK -9

```
#include <iostream>
#include <iomanip>
```

- LINGESH.S
- (1924260290)

```
#include <string>
#include <sstream>
using namespace std;

// Fare values are stored in paise
const long long MIN_BALANCE = 5000;
const int PEAK_SURCHARGE_PERCENT = 20;

bool isPeakHour(int hour)
{
   return (hour >= 8 && hour < 10) ||
          (hour >= 17 && hour < 20);
}

long long calculateFare(int zone, int hour)
{
   if (hour < 0 || hour > 23)
      return -1;

   long long baseFare;

   switch (zone)
   {
      case 1: baseFare = 1000; break; // Rs. 10
      case 2: baseFare = 2000; break; // Rs. 20
      case 3: baseFare = 3000; break; // Rs. 30
```

```
        default: return -1;
    }

    if (isPeakHour(hour))
        baseFare += baseFare * PEAK_SURCHARGE_PERCENT / 100;

    return baseFare;
}

bool canTravel(long long balance, long long fare)
{
    return fare >= 0 &&
           balance >= fare &&
           balance - fare >= MIN_BALANCE;
}

bool recharge(long long &balance, long long amount)
{
    if (amount < 0 || amount > 1000000000000LL - balance)
        return false;

    balance += amount;
    return true;
}

void printMoney(long long paise)
```

```
{
    cout << "Rs. " << paise / 100 << "."
         << setw(2) << setfill('0') << paise % 100
         << setfill(' ');
}

int main()
{
    cout << "===== FARE LOGIC TESTING =====\n\n";

    struct Test
    {
        int zone;
        int hour;
        long long expected;
    };

    Test tests[] = {
        {1, 7, 1000},    // Inner, off-peak
        {2, 8, 2400},    // Middle, peak
        {3, 12, 3000},   // Outer, off-peak
        {1, 17, 1200},   // Inner, peak
        {2, 21, 2000},   // Middle, off-peak
        {1, 8, 1200},    // Exactly 8:00
        {1, 10, 1000},   // Exactly 10:00
        {1, 17, 1200},   // Exactly 17:00
```

```
    {1, 20, 1000},   // Exactly 20:00
    {4, 12, -1}      // Invalid zone
};

int count = sizeof(tests) / sizeof(tests[0]);
int passed = 0;

cout << left << setw(8) << "Zone"
     << setw(8) << "Hour"
     << setw(16) << "Expected"
     << setw(16) << "Actual"
     << "Result\n";

for (int i = 0; i < count; i++)
{
    long long actual =
        calculateFare(tests[i].zone, tests[i].hour);

    cout << left << setw(8) << tests[i].zone
         << setw(8) << tests[i].hour;

    if (tests[i].expected == -1)
        cout << setw(16) << "Invalid";
    else
    {
        ostringstream out;
```

```
        out << tests[i].expected / 100 << "."
            << setw(2) << setfill('0')
            << tests[i].expected % 100;
        cout << setw(16) << out.str();
    }

    if (actual == -1)
        cout << setw(16) << "Invalid";
    else
    {
        ostringstream out;
        out << actual / 100 << "."
            << setw(2) << setfill('0') << actual % 100;
        cout << setw(16) << out.str();
    }

    bool ok = (actual == tests[i].expected);
    cout << (ok ? "PASS" : "FAIL") << '\n';

    if (ok)
        passed++;
}

cout << "\nTests passed: " << passed
     << "/" << count << '\n';
```

- LINGESH.S
- (1924260290)

```
// Test minimum balance
long long balance = 6000;
long long fare = 1000;

cout << "\nMinimum balance test: ";
if (canTravel(balance, fare))
{
   balance -= fare;
   cout << "PASS (remaining balance = ";
   printMoney(balance);
   cout << ")\n";
}
else
   cout << "FAIL\n";

// Test negative recharge
cout << "Negative recharge test: ";
cout << (recharge(balance, -500) ? "FAIL" : "PASS")
    << '\n';

// Test letters entered instead of an hour
string input = "abc";
istringstream stream(input);
int hour;

cout << "Invalid hour input test: ";
```

```
    if (!(stream >> hour))
        cout << "PASS (letters rejected)\n";
    else
        cout << "FAIL\n";

    // Five journeys: calculate total manually
    int zones[] = {1, 2, 3, 1, 2};
    int hours[] = {7, 8, 12, 17, 21};
    long long total = 0;

    for (int i = 0; i < 5; i++)
        total += calculateFare(zones[i], hours[i]);

    cout << "\nFive-journey total: ";
    printMoney(total);
    cout << '\n';

    return 0;
}
```

TASK -10

```
#include <iostream>
#include <iomanip>
#include <climits>
#include <limits>
```

- LINGESH.S
- (1924260290)

```
using namespace std;


int main()
{
   const long long farePaise = 1001; // Rs. 10.01
   const double fareRupees = 10.01;

   long long journeyCounts[] = {
      20000, 500000, 3000000
   };

   cout << fixed << setprecision(10);
   cout << "FARE TOTAL COMPARISON\n\n";

   for (long long n : journeyCounts)
   {
      double doubleTotal = 0.0;
      for (long long i = 0; i < n; ++i)
         doubleTotal += fareRupees;

      long long paiseTotal = n * farePaise;
      double exactRupees = paiseTotal / 100.0;

      cout << "Journeys: " << n << '\n';
      cout << "Double total: Rs. "
           << doubleTotal << '\n';
```

```
    cout << "Integer-paise total: Rs. "
         << exactRupees << '\n';
    cout << "Difference: Rs. "
         << doubleTotal - exactRupees << "\n\n";
}

// Check whether the total fits in int
long long totalPaise = 3000000LL * farePaise;

cout << "INTEGER OVERFLOW CHECK\n";
cout << "Network total: " << totalPaise
     << " paise\n";
cout << "INT_MAX: " << INT_MAX << '\n';

if (totalPaise > INT_MAX)
    cout << "Result: int is NOT sufficient.\n";
else
    cout << "Result: int is sufficient.\n";

cout << "LLONG_MAX: " << LLONG_MAX << '\n';

// Compare storage sizes on this system
cout << "\nDATA TYPE SIZES\n";
cout << "int: " << sizeof(int) << " bytes\n";
cout << "long long: " << sizeof(long long)
     << " bytes\n";
```

```
    cout << "double: " << sizeof(double)
         << " bytes\n";

    // Safely demonstrate int range checking
    if (totalPaise > INT_MAX)
        cout << "\nChecked result: use long long "
             << "for the network total.\n";

    return 0;
}
```

TASK -11

```
#include <iostream>
#include <iomanip>
#include <limits>
using namespace std;

// Fare configuration: all money values are in paise
constexpr int ZONE_COUNT = 3;
constexpr long long BASE_FARE[ZONE_COUNT] = {
    1000, 2000, 3000
};

constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE = 5000;
```

- LINGESH.S
- (1924260290)

```
// Check whether the hour is within peak periods
bool isPeakHour(int hour)
{
   return (hour >= 8 && hour < 10) ||
          (hour >= 17 && hour < 20);
}

// Calculate fare for a zone
long long calculateFare(int zone, int hour)
{
   if (zone < 1 || zone > ZONE_COUNT ||
      hour < 0 || hour > 23)
      return -1;

   long long fare = BASE_FARE[zone - 1];

   if (isPeakHour(hour))
      fare += fare * PEAK_SURCHARGE_PERCENT / 100;

   return fare;
}

// SmartCard class protects its balance
class SmartCard
{
```

- LINGESH.S
- (1924260290)

```
private:
    long long balance;

public:
    explicit SmartCard(long long initialBalance)
        : balance(initialBalance >= 0 ? initialBalance : 0) {}

    bool canTravel(long long fare) const
    {
        return fare >= 0 &&
               balance >= fare &&
               balance - fare >= MIN_BALANCE;
    }

    bool deduct(long long fare)
    {
        if (!canTravel(fare))
            return false;

        balance -= fare;
        return true;
    }

    long long getBalance() const
    {
        return balance;
```

```
    }

    bool recharge(long long amount)
    {
        if (amount < 0 ||
            amount > numeric_limits<long long>::max() - balance)
            return false;

        balance += amount;
        return true;
    }
};

void printMoney(long long paise)
{
    cout << "Rs. " << paise / 100 << "."
         << setw(2) << setfill('0') << paise % 100
         << setfill(' ');
}

int main()
{
    cout << "===== METRO FARE SYSTEM =====\n";

    SmartCard card(10000); // Rs. 100
```

- LINGESH.S
- (1924260290)

```
struct Journey
{
    int zone;
    int hour;
};

Journey journeys[] = {
    {1, 7},   // Inner, off-peak
    {2, 8},   // Middle, peak
    {3, 12},  // Outer, off-peak
    {1, 17},  // Inner, peak
    {2, 20}   // Middle, off-peak
};

const int count = sizeof(journeys) / sizeof(journeys[0]);
long long totalCollected = 0;

for (int i = 0; i < count; ++i)
{
    long long fare = calculateFare(
        journeys[i].zone, journeys[i].hour);

    cout << "\nJourney " << i + 1 << ": ";

    if (fare < 0)
    {
```

```
        cout << "Invalid journey\n";
        continue;
    }

    cout << "Fare ";
    printMoney(fare);

    if (card.deduct(fare))
    {
        totalCollected += fare;
        cout << " - Approved";
    }
    else
    {
        cout << " - Denied (minimum balance rule)";
    }

    cout << "\nRemaining balance: ";
    printMoney(card.getBalance());
    cout << '\n';
}

cout << "\nTotal fare collected: ";
printMoney(totalCollected);

cout << "\nFinal card balance: ";
```

```
    printMoney(card.getBalance());

    cout << "\n\nPeak-hour boundary tests:\n";
    cout << "08:00: " << boolalpha << isPeakHour(8) << '\n';
    cout << "10:00: " << isPeakHour(10) << '\n';
    cout << "17:00: " << isPeakHour(17) << '\n';
    cout << "20:00: " << isPeakHour(20) << '\n';

    return 0;
}
```

TASK -12

```
#include <iostream>
#include <array>
#include <iomanip>
using namespace std;

enum class Zone { Inner, Middle, Outer, Airport };

constexpr array<long long, 4> BASE_FARE = {
    1000, 2000, 3000, 6000
}; // Paise: Rs. 10, 20, 30, 60

constexpr int PEAK_PERCENT = 20;
```

- LINGESH.S
- (1924260290)

```
constexpr int WEEKEND_DISCOUNT = 10;
constexpr long long MIN_BALANCE = 5000;

// Monthly pass allows unlimited journeys in two zones
class MonthlyPass {
private:
   Zone zone1, zone2;
   bool valid;

public:
   MonthlyPass(Zone a, Zone b, bool active = true)
      : zone1(a), zone2(b), valid(active) {}

   bool covers(Zone z) const {
      return valid && (z == zone1 || z == zone2);
   }
};

bool isPeak(int hour) {
   return (hour >= 8 && hour < 10) ||
          (hour >= 17 && hour < 20);
}

long long calculateFare(Zone zone, int hour, bool weekend) {
   int index = static_cast<int>(zone);
```

```
    if (index < 0 || index >= 4 || hour < 0 || hour > 23)
        return -1;

    long long fare = BASE_FARE[index];

    // Weekend discount and peak surcharge never combine
    if (weekend)
        fare -= fare * WEEKEND_DISCOUNT / 100;
    else if (isPeak(hour))
        fare += fare * PEAK_PERCENT / 100;

    return fare;
}

class SmartCard {
private:
    long long balance;

public:
    explicit SmartCard(long long amount) : balance(amount) {}

    bool travel(Zone zone, int hour, bool weekend,
                const MonthlyPass& pass) {
        if (hour < 0 || hour > 23 ||
            static_cast<int>(zone) < 0 ||
            static_cast<int>(zone) >= 4) {
```

```
        cout << "Invalid journey\n";
        return false;
    }

    // A covered zone is free with a valid pass
    if (pass.covers(zone)) {
        cout << "Monthly pass: Free journey\n";
        return true;
    }

    long long fare = calculateFare(zone, hour, weekend);

    if (balance - fare < MIN_BALANCE) {
        cout << "Denied: minimum balance required\n";
        return false;
    }

    balance -= fare;
    cout << "Fare: Rs. " << fare / 100 << "."
         << setw(2) << setfill('0') << fare % 100
         << setfill(' ') << '\n';
    return true;
}

void showBalance() const {
    cout << "Balance: Rs. " << balance / 100 << "."
```

```
        << setw(2) << setfill('0') << balance % 100
        << setfill(' ') << '\n';
    }
};

int main() {
    SmartCard card(20000); // Rs. 200

    MonthlyPass pass(Zone::Inner, Zone::Middle);

    cout << "METRO FARE SYSTEM\n";

    cout << "\n1. Inner zone, weekday off-peak\n";
    card.travel(Zone::Inner, 11, false, MonthlyPass(
        Zone::Airport, Zone::Outer));
    card.showBalance();

    cout << "\n2. Middle zone, weekday peak\n";
    card.travel(Zone::Middle, 8, false, MonthlyPass(
        Zone::Airport, Zone::Outer));
    card.showBalance();

    cout << "\n3. Airport zone, weekend\n";
    card.travel(Zone::Airport, 12, true, pass);
    card.showBalance();
```

```
    cout << "\n4. Inner zone, weekend with pass\n";
    card.travel(Zone::Inner, 12, true, pass);
    card.showBalance();

    cout << "\n5. Outer zone, weekend\n";
    card.travel(Zone::Outer, 12, true, pass);
    card.showBalance();

    return 0;
}
```